MACHINE LEARNING PROJECT

Credit Card Fraud Detection

Random Forest model deployed with Streamlit



By Aryan Rawat · Python · scikit-learn · Streamlit



Made with GAMMA

01 · THE PROBLEM

Fraud detection is a real-time decision under extreme imbalance.

Given a payment's amount and time, decide whether it is fraud or genuine.



High cost

Card fraud causes huge losses for banks and customers.



Real time

The decision must happen while the payment is being evaluated.



Rare events

Frauds are very rare, so the data is highly imbalanced.



End to end

Analyse data, train Random Forest, then serve predictions through a web app.

Made with GAMMA

02 · DATASET

The dataset is broad, anonymised, and overwhelmingly legitimate.

284,807
transactions

492
frauds · 0.172%

31
columns

2 days
September 2013, EU cardholders

- Time: seconds since the first transaction
- V1–V28: PCA-transformed features (anonymised)
- Amount: transaction value · Class: 1 = fraud, 0 = genuine
- `Source: Kaggle, mlg-ulb/creditcardfraud`

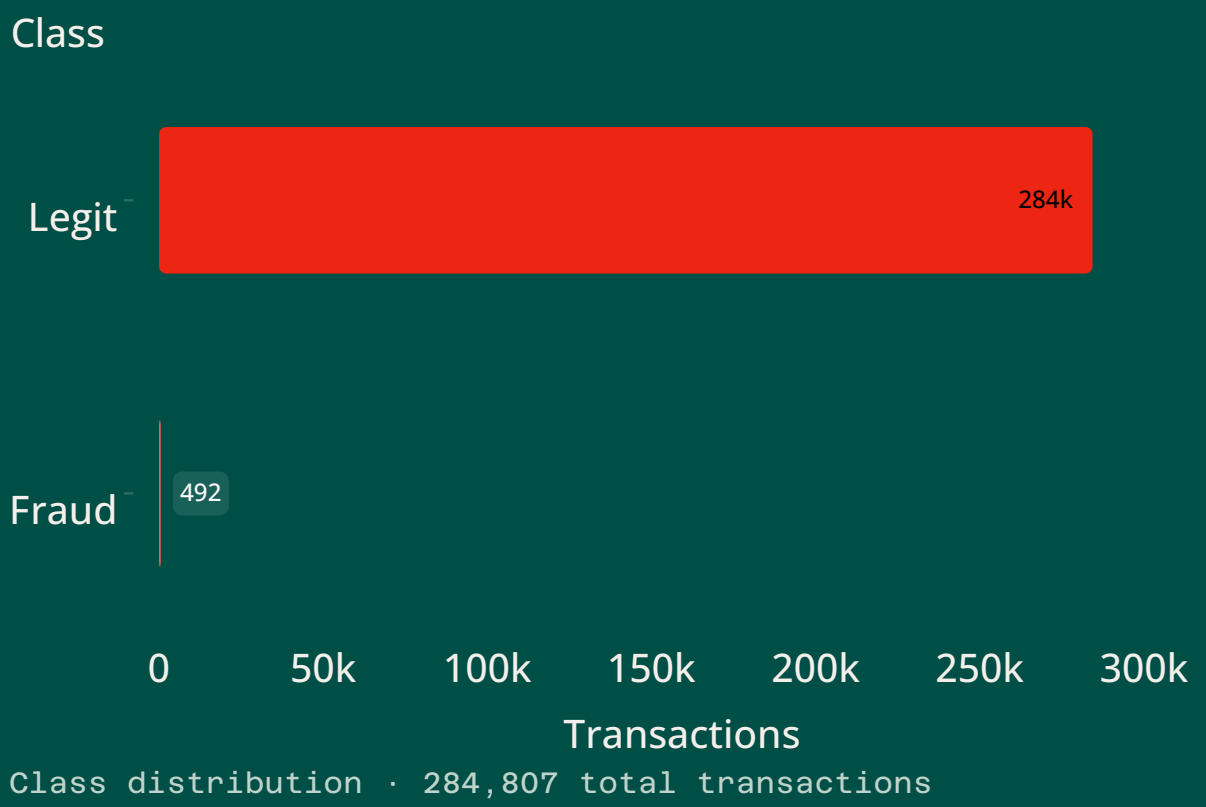
Made with GAMMA

03 · DATA ANALYSIS

Accuracy can look excellent while missing every fraud.

Only 0.172% of rows are fraud. Predicting “genuine” every time gives roughly 99.8% accuracy, so accuracy alone is misleading.

USE PRECISION · RECALL · F1



Class distribution · 284,807 total transactions

Made with GAMMA

04 · MODEL PIPELINE

From raw transactions to a saved Random Forest.

01

Load CSV

Read the transaction data.

02

Explore & clean

Inspect distributions and prepare inputs.

03

Scale

Apply RobustScaler to Amount / Time.

04

Train-test split

Hold out data for evaluation.

05

Random Forest

Learn non-linear fraud patterns.

06

Save .pkl

Persist the model for deployment.

Why Random Forest? Handles non-linear patterns, robust to noise, gives feature importance.

Made with GAMMA

05 · MODEL RESULTS

Random Forest keeps fraud coverage high while cutting false alarms.

0.83
Precision

0.82
Recall

0.82
F1-score

99.94%
Accuracy

Model	Frauds caught	False alarms	Fraud precision
Logistic Regression	90 of 98	1,347	0.06
Random Forest	80 of 98	16	0.83

Random Forest wins: almost the same fraud coverage with far fewer false alarms, 16 vs 1,347.

Made with GAMMA

06 · CONFUSION MATRIX

The trade-off is visible in the errors.

Random Forest catches 80 frauds and misses 18, while raising only 16 false alarms out of 56,864 legitimate transactions.

FRAUD PRECISION 0.83 · RECALL 0.82

Actual / predicted	Legit	Fraud
Legit	56,848	16 · false alarms
Fraud	18 · missed	80 · caught

Classification report · Random Forest test set

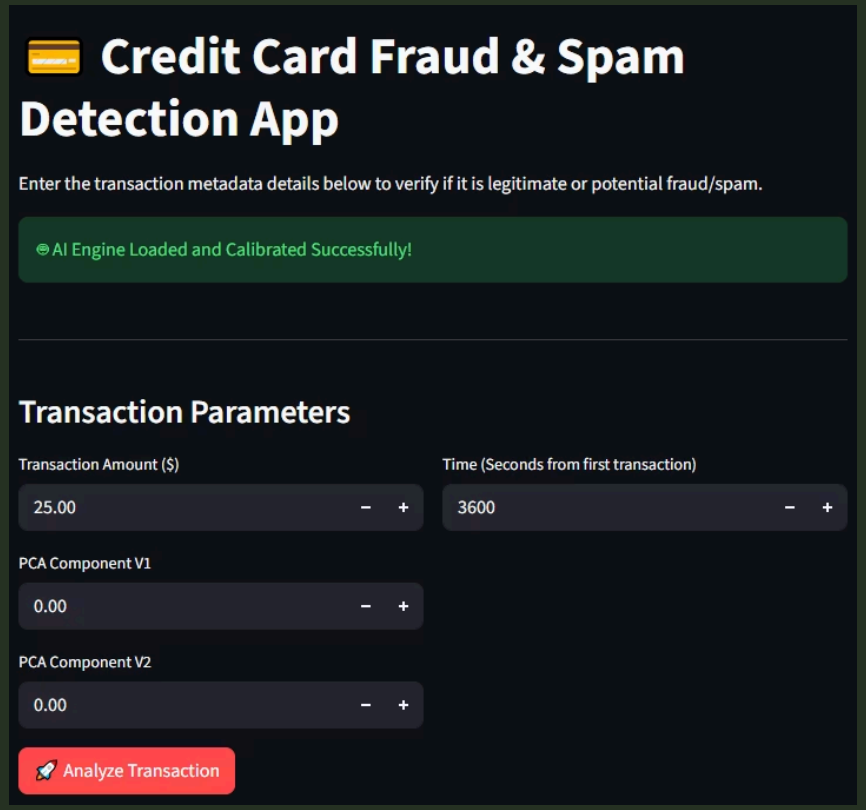
Class	Precision	Recall	F1-score	Support
0	1.00	1.00	1.00	56864
1	0.83	0.82	0.82	98

Made with GAMMA

07 · DEPLOYMENT

Streamlit turns the trained model into a usable decision tool.

- Model + scaler saved with joblib (.pkl)
- User enters Time, Amount and key features
- App returns Fraud / Legit with probability
- `streamlit run app.py`



Evidence asset · Streamlit interface

Made with GAMMA

08 · PROJECT FILES & NEXT STEPS

The project is packaged for reuse, tuning, and monitoring.

GitHub repository

Detection.ipynb
app.py
credit_card_rf_model.pkl
robust_scaler.pkl
requirements.txt
README.md

Future work

- Tune hyperparameters and threshold
- Try XGBoost / SMOTE
- Add CSV batch scoring
- Monitor drift

Thank you

Made with GAMMA